

Computer Networks

Emad Ezzadeen

30077582

Module Code: CS3S661

Source code repository Link:

<https://github.com/emadezzz/distance-vector-routing>

Table of Contents

<i>Introduction.....</i>	<i>2</i>
<i>Background and Context.....</i>	<i>3</i>
<i>Simulation Design.....</i>	<i>5</i>
<i>Operation of the Simulation</i>	<i>7</i>
<i>Conclusion and Future Work.....</i>	<i>9</i>
<i>References</i>	<i>10</i>

Figure 1: Network topology used in the simulation showing routers A, B, C, D, and E with associated link costs	6
Figure 2: Initial routing table of Router A showing direct neighbours and initial costs.	7
Figure 3: Final routing table of Router A after convergence showing optimal paths to all destinations.....	8

Introduction

Computer networks rely on routing mechanisms to allow data to move between different devices efficiently. One of the key components in this process is the routing table, which stores information about possible paths to different destinations. Without this, communication across networks would not be possible in a structured or reliable way.

This project focuses on simulating the process of building a routing table using a distance vector routing protocol such as RIP. The aim is not to fully implement a real protocol, but to demonstrate how routers exchange information and gradually discover the best paths in a network over time.

A simple application was developed using Python, where multiple routers run as separate instances and communicate with each other through UDP sockets. Each router shares its routing information with its neighbours and updates its table when a better route is identified during the simulation.

This simulation provides a clear and practical way to understand how routing tables are constructed and updated in real network environments, while keeping the implementation simple and easy to follow.

Background and Context

Computer networks depend on routing protocols to allow data to move between different devices efficiently. A routing protocol defines how routers communicate and how the best path is selected between a source and a destination. In large and dynamic networks, routing becomes more complex, and manual configuration is not practical. For this reason, dynamic routing protocols are commonly used to automatically update routing tables when the network changes (Masruroh et al., 2016). This becomes especially important in modern networks where devices are frequently added or removed, and paths may change over time, requiring continuous updates.

Routing tables play a central role in this process. Each router maintains a table that stores information about possible destinations and the best next hop to reach them. These tables are used to forward data packets correctly across the network. If the routing table is incorrect or outdated, data packets may be lost, delayed, or sent through inefficient paths, which directly affects network performance (Journal, 2015). Therefore, routing protocols must continuously exchange information to maintain accurate and efficient paths and ensure reliable communication between nodes.

Distance vector routing is one of the simplest approaches used in routing protocols. In this method, each router shares its routing table with its neighbours. The router then updates its own table based on the information received. This process is repeated until all routers reach a stable state, where the best paths are known. The Routing Information Protocol (RIP) is a well known example of a distance vector protocol. It works by sending routing updates at regular intervals and uses hop count as a metric to determine the best path (Journal, 2015). Because of its simplicity, RIP is easy to implement and understand, but it also has limitations, such as slow convergence, limited scalability, and restrictions on network size.

The process of finding the best path in distance vector routing is based on the Bellman Ford algorithm. This algorithm calculates the shortest path by comparing different routes and updating the cost step by step. Each router updates its knowledge based only on information received from its neighbours, which makes the approach fully distributed. This is one of the main reasons why distance vector routing is widely used in networking, especially in simpler environments where ease of implementation is important (Parimala et al., 2021).

In contrast, link state routing protocols such as OSPF use a different approach. Instead of sharing full routing tables, routers exchange information about the state of their links. Each router then builds a complete map of the network and calculates the shortest path independently. This allows faster convergence and more accurate routing decisions in large networks. However, this approach also requires more processing power and generates more control traffic, which can increase complexity and resource usage (Journal, 2015).

In real world networks, both approaches are used depending on the network size and requirements. Distance vector protocols like RIP are suitable for smaller and simpler networks due to their low complexity and ease of implementation. On the other hand, link state protocols like OSPF are preferred in larger and more complex environments where performance, scalability, and faster convergence are more important, especially in enterprise level networks.

Simulation Design

The simulation was designed to demonstrate how routing tables are built using a distance vector approach in a simple and controlled environment. The main goal was to keep the system clear and easy to understand, while still showing how routers communicate and update their routing information. The design avoids unnecessary complexity so that the main routing behaviour can be observed clearly during execution and easily explained.

Python was chosen as the programming language because it provides a simple and readable structure, which makes it suitable for building a network simulation. It allows the logic of the program to be written in a clear way, which helps when explaining the system in the report. In addition, Python includes built-in support for socket programming, allowing communication between different router instances without the need for additional libraries. This makes the development process faster and allows more focus on the routing logic instead of technical details.

UDP sockets were used for communication between routers. This choice was made because UDP is lightweight and does not require connection setup, which reduces overhead during communication. Since the simulation runs in a local and controlled environment, there is no need for complex reliability mechanisms such as retransmission or error correction. This keeps the implementation simple while still demonstrating how routers exchange information in a clear way.

The routing logic is based on the Bellman Ford approach, which is commonly used in distance vector routing. Each router calculates the cost to reach a destination by combining its link cost with the cost received from its neighbours. This process is repeated continuously, allowing routers to gradually discover better paths. Over time, the routing tables become more accurate as better routes are found and updated. The use of this approach makes the simulation closer to how real distance vector protocols operate.

A simple network topology was defined in the configuration file, where five routers are connected with different link costs. The number of routers was chosen to provide enough complexity to demonstrate routing behaviour, while still keeping the system easy to follow. Each router is connected to a limited number of neighbours, which helps to clearly show how updates move through the network and how information propagates between nodes.

Each router runs as a separate instance of the program and communicates with others using predefined ports. This design allows multiple routers to operate at the same time, simulating a distributed network environment. It also reflects how real routers operate independently while still being part of the same network and sharing information continuously.

Overall, the design focuses on clarity and simplicity, while still capturing the key behaviour of distance vector routing and inter router communication in a practical and easy to understand way.

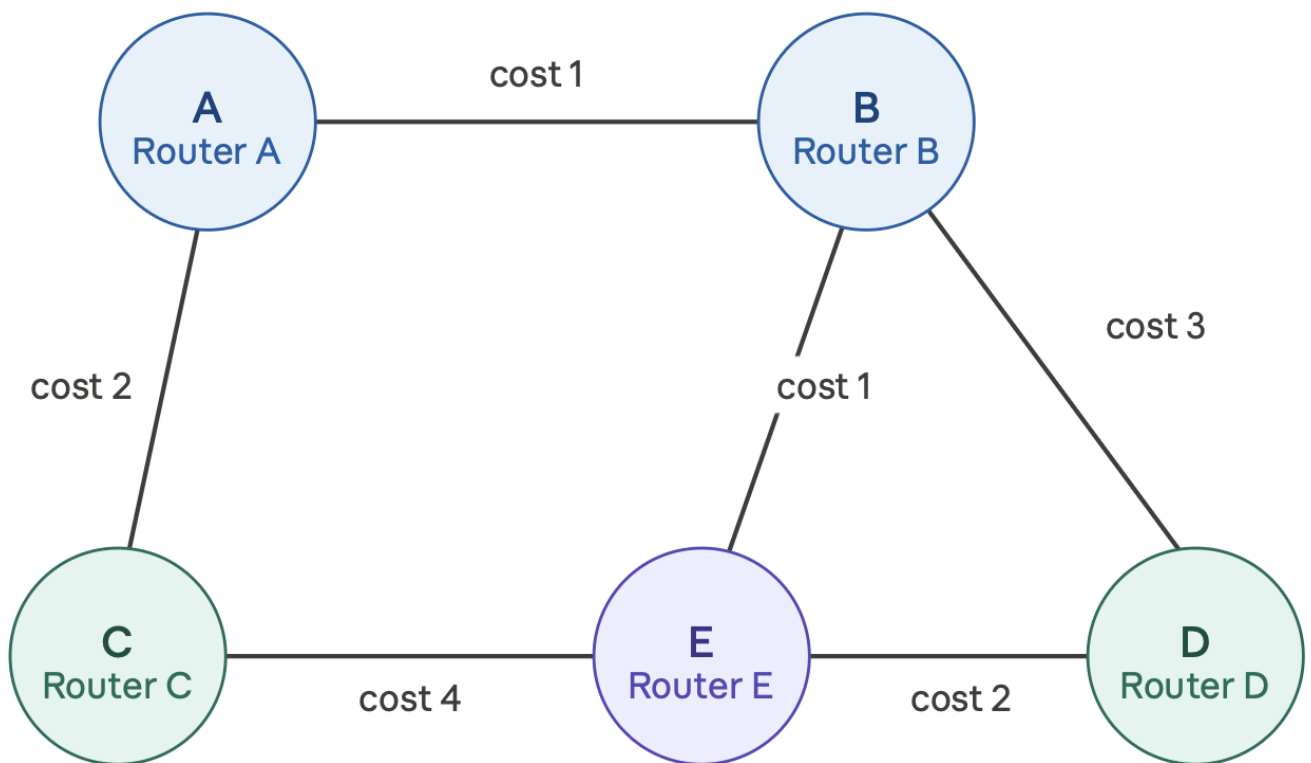


Figure 1: Network topology used in the simulation showing routers A, B, C, D, and E with associated link costs

Operation of the Simulation

The simulation is based on running multiple router instances that communicate with each other and update their routing tables over time. Each router operates independently but follows the same logic, which allows the network to gradually converge to the best paths. This setup reflects how real routers work in a distributed environment, where no single device has full control of the network and decisions are made locally.

The configuration file defines the network topology, including the routers, their ports, and the cost between neighbours. Each router is assigned a unique port, and communication is performed locally using these ports. This setup allows multiple routers to run at the same time and exchange messages. The topology is intentionally kept simple so that the behaviour of the routing process can be clearly observed and easily understood during execution.

When a router starts, it initializes its routing table with two types of entries. The first entry is the route to itself with a cost of zero. The second type includes direct neighbours, where the cost is taken from the predefined topology. At this stage, the router only knows about its immediate connections and has no information about the rest of the network.

Router A - Routing Table		
Dest	Cost	Next Hop
A	0	A
B	1	B
C	2	C

```
[16:51:08] Router A: Starting on port 5001. Neighbours: {'B': 1, 'C': 2}
[16:51:08] Router A: Router running. Press Ctrl+C to stop.

[16:51:09] Router A: Sent update to: ['B', 'C']
[16:51:14] Router A: Sent update to: ['B', 'C']
[16:51:19] Router A: Sent update to: ['B', 'C']
[16:51:24] Router A: Sent update to: ['B', 'C']
[16:51:29] Router A: Sent update to: ['B', 'C']
[16:51:34] Router A: Sent update to: ['B', 'C']
[16:51:39] Router A: Sent update to: ['B', 'C']
```

Figure 2: Initial routing table of Router A showing direct neighbours and initial costs.

The communication between routers is handled using UDP sockets. Each router runs two parallel processes using threads. One thread is responsible for sending updates at regular intervals, while the other listens for incoming messages. This allows routers to send and receive information at the same time without blocking execution. The use of UDP keeps the communication simple and fast within the simulation environment.

Each router periodically sends its routing table to its neighbours. These updates contain the current known cost to each destination. When a router receives an update, it stores the information and uses it to recalculate its own routing table. This process happens continuously as long as the simulation is running and forms the core of the routing behaviour.

The update process follows a simple rule. For each destination, the router compares the current cost with a new cost calculated through a neighbour. This new cost is the sum of the link cost to the neighbour and the cost advertised by that neighbour. If the new cost is lower, or if the current route depends on that neighbour and has changed, the routing table is updated with the new value and the corresponding next hop. This ensures that routing decisions improve over time as better paths are discovered.

The simulation continues this process repeatedly. As routers exchange information, the routing tables become more accurate over time. Routes that were previously unknown are discovered, and existing routes may be improved if a shorter path is found. Eventually, all routers reach a stable state where no better routes can be found. This behaviour represents the convergence of the routing protocol and shows that the network has reached a consistent state.

Router A – Routing Table		
Dest	Cost	Next Hop
A	0	A
B	1	B
C	2	C
D	4	B
E	2	B

Figure 3: Final routing table of Router A after convergence showing optimal paths to all destinations

The output of the simulation clearly shows this process. Each router prints its routing table whenever a change occurs, making it easy to observe how routes are discovered and updated step by step. This makes the simulation useful for understanding how routing tables are built in a distributed network and how routers cooperate to find optimal paths.

Conclusion and Future Work

This project demonstrated how routing tables can be built using a distance vector approach in a simple and practical way. The simulation showed how routers exchange information and gradually discover the best paths through continuous updates. The behaviour observed during execution reflects the core idea used in real routing protocols such as RIP, where routers rely on periodic updates to maintain accurate routing information.

The implementation focused on clarity rather than completeness. It successfully showed communication between routers, the update process, and how routing decisions are made based on cost. This makes the simulation useful for understanding the basic operation of routing in distributed networks and provides a clear example of how routing logic works in practice.

However, the simulation has some limitations. It does not include advanced features such as route timers, loop prevention techniques, or failure detection. In real networks, these mechanisms are important to ensure stability and avoid routing problems such as routing loops or slow convergence. The use of UDP without reliability control also means that message delivery is assumed to be perfect, which is not always the case in real network environments.

There are several ways this project could be improved. Additional features such as handling link failures, implementing route expiration, or adding loop prevention methods could make the simulation more realistic. For example, techniques that prevent incorrect routing updates could be introduced to improve stability. Security aspects could also be considered, such as validating routing updates to prevent incorrect or malicious information from being accepted.

Overall, the project provides a clear and simple representation of how routing tables are constructed and updated in a network environment. It also shows how a basic simulation can be extended to reflect more realistic networking scenarios in the future.

References

Journal, I. (2015) 'RIP, OSPF, EIGRP ROUTING PROTOCOLS'. *IJRCAR* . Available at: https://www.academia.edu/14536341/RIP_OSPF_EIGRP_ROUTING_PROTOCOLS.

Masruroh, S.U., Robby, F. and Hakiem, N. (2016) *Performance evaluation of routing protocols RIPng, OSPFv3, and EIGRP in an IPv6 network*. Available at: <https://ieeexplore.ieee.org/abstract/document/7905699> (Accessed: 13 February 2022).

Parimala, M., Broumi, S., Prakash, K. and Topal, S. (2021) 'Bellman–Ford algorithm for solving shortest path problem of a network under picture fuzzy environment'. *Complex & Intelligent Systems* 7(5), pp. 2373–2381. doi: <https://doi.org/10.1007/s40747-021-00430-w>.